

미션 17 요약 보고서

손글씨 숫자 인식 Streamlit 서비스

팀/이름: 1팀 안호성

기본 모델: GitHub ONNX 모델 저장소의 MNIST ONNX 모델

핵심 기술: Streamlit, streamlit-drawable-canvas, ONNX

Runtime, Docker

핵심 기능: 대형 입력 캔버스, 전처리 이미지, 확률 차트, 저장소 갤러리

핵심 요약

- 브라우저에서 직접 숫자를 그리고 즉시 예측 결과를 확인할 수 있다.
- 모델은 자동 다운로드되며 ONNX Runtime 세션은 캐싱된다.
- 손그림과 예측 결과는 저장소에 누적되어 다시 확인할 수 있다.
- Dockerfile을 포함해 서비스 배포 경로까지 정리했다.

1. 프로젝트 개요

이번 미션은 사용자가 웹에서 직접 숫자를 그리고, ONNX 기반 MNIST 분류 모델이 예측 결과를 반환하는 AI 서비스를 구현하는 과제이다.

단순 모델 구현에 그치지 않고, 전처리 시각화와 결과 저장소까지 포함한 서비스 형태로 확장하는 것을 목표로 했다.

2. 서비스 구성

- 입력 캔버스: streamlit-drawable-canvas로 숫자를 직접 그림
- 전처리 이미지: crop, pad, resize를 거친 28x28 입력 이미지 표시
- 추론 결과: 0~9 확률 분포를 막대 차트로 시각화
- 이미지 저장소: 손그림과 예측 결과를 로컬 파일 + 메타데이터로 저장
- 레이아웃: 상단 요약 카드 + 좌측 대형 캔버스 + 우측 결과 패널로 구성

3. 코드 설명

- download_model_if_needed(): 모델 파일이 없을 때만 GitHub ONNX 모델 저장소에서 다운로드
- load_onnx_session(): st.cache_resource로 ONNX Runtime 세션 캐싱
- preprocess_canvas_image(): 캔버스 입력을 MNIST 입력 형식 [1,1,28,28]로 변환
- run_inference(): softmax 기반 확률 계산 및 최고 확률 숫자 반환
- save_gallery_entry(): 원본 이미지, 전처리 이미지, 예측 결과를 saved_images에 저장
- UI 스타일: .streamlit/config.toml과 assets/style.css로 화면 테마와 카드형 스타일 분리

4. Docker 구성

- 베이스 이미지는 python:3.11-slim을 사용했다.
- requirements.txt를 먼저 설치한 뒤 앱 소스를 복사하도록 구성했다.
- 컨테이너 실행 시 streamlit run app.py --server.address=0.0.0.0
--server.port=8501 명령으로 서비스가 열린다.

5. 실행 방법

로컬 실행

```
cd Mission_Archive/Codeit_Mission/미션17_1팀_안호성
pip install -r requirements.txt
streamlit run app.py
```

Docker 실행

```
docker build -t mnist-onnx-streamlit:latest .
docker run --rm -p 8501:8501 mnist-onnx-streamlit:latest
```

6. Docker Hub URL

실제 업로드한 Docker Hub 저장소 URL은 아래와 같다.

<https://hub.docker.com/r/ahnhs2k/mnist-onnx-streamlit>

7. Docker Hub 배포 절차

- docker login
- docker tag mnist-onnx-streamlit:latest
ahnhs2k/mnist-onnx-streamlit:latest
- docker push ahnhs2k/mnist-onnx-streamlit:latest

8. 결론

- 입력 캔버스, 전처리 이미지, 확률 차트, 저장소 갤러리를 포함한 완전한 숫자 인식 웹 서비스를 구현했다.
- 모델 다운로드와 ONNX 세션 캐싱을 통해 반복 실행 효율도 고려했다.
- Dockerfile까지 포함해 로컬 실행뿐 아니라 배포 가능한 형태로 프로젝트를 정리했다.